
PROYECTO 2 – IPC2-AcadNet

202404856 – Angel Emanuel Rodriguez Corado

Resumen

El presente ensayo aborda la digitalización de la educación, un tema de gran vigencia tanto a nivel nacional como internacional, especialmente tras la pandemia de COVID-19 que aceleró la adopción de tecnologías en el ámbito educativo. La digitalización representa una transformación profunda en la forma de enseñar y aprender, permitiendo el acceso a recursos y plataformas virtuales que antes eran limitados o inexistentes. Existen diversas posturas respecto a este fenómeno: algunos expertos destacan los beneficios técnicos y económicos, como la reducción de costos y la ampliación de cobertura educativa; otros advierten sobre la brecha digital y los desafíos sociales y ambientales, como el acceso desigual a la tecnología y el aumento en el consumo energético. A nivel técnico, la digitalización impulsa la innovación y el desarrollo de nuevas metodologías; sin embargo, también exige una actualización constante de docentes y estudiantes. En conclusión, la digitalización educativa es un proceso irreversible que requiere políticas inclusivas y sostenibles para maximizar sus beneficios y minimizar sus impactos negativos.

Palabras clave

Programación Orientada a Objetos, Python, archivos XML, Flask, Django

Abstract

This essay addresses the digitalization of education, a highly relevant topic both nationally and internationally, particularly in the wake of the COVID-19 pandemic, which accelerated the adoption of technology in the educational sphere. Digitalization represents a profound transformation in the ways teaching and learning take place, enabling access to virtual platforms and resources that were previously limited or non-existent. There are varying perspectives on this phenomenon: some experts emphasize its technical and economic advantages, such as cost reduction and expanded educational coverage, while others highlight concerns regarding the digital divide and the associated social and environmental challenges, including unequal access to technology and increased energy consumption. From a technical standpoint, digitalization drives innovation and the development of new methodologies; however, it also demands continuous adaptation from both educators and students. In conclusion, educational digitalization is an irreversible process that calls for inclusive and sustainable policies in order to maximize its benefits and mitigate its adverse effects.

Keywords

Object-Oriented Programming, Python, XML files, Flask, Django

Introducción

En la actualidad, la digitalización de los procesos académicos se ha vuelto esencial para mejorar la eficiencia, la transparencia y la accesibilidad en la educación. Este ensayo analiza el desarrollo del sistema IPC2–AcadNet, una solución web que integra tecnologías como Django, Flask y Graphviz para automatizar la carga, visualización y reporte de horarios y calificaciones estudiantiles. La relevancia de este sistema radica en su capacidad para optimizar la gestión educativa mediante interfaces intuitivas y representaciones gráficas precisas, facilitando la toma de decisiones por parte de tutores y estudiantes. ¿Cómo impacta esta herramienta en la experiencia educativa? ¿Qué desafíos técnicos y pedagógicos conlleva su implementación? Este ensayo busca responder a estas interrogantes, presentando aportes técnicos y reflexivos sobre la integración de herramientas digitales en el entorno educativo actual, marcando un precedente sobre cómo la tecnología puede transformar positivamente la educación superior.

Desarrollo del tema

La implementación del sistema IPC2–AcadNet representa una solución integral orientada a la gestión académica de tutores y estudiantes mediante tecnologías web y el uso de estructuras de datos avanzadas. Este proyecto se enfoca en facilitar la carga, procesamiento y visualización de información académica, aprovechando herramientas como Django, Flask, Graphviz y el uso de estructuras tipo matriz dispersa.

a) Uso de estructuras de datos personalizadas: Matriz dispersa

Uno de los pilares del proyecto es la implementación de una matriz dispersa como

estructura de datos para almacenar notas. Esta permite representar relaciones entre múltiples dimensiones (actividad y estudiante) sin desperdiciar memoria en valores nulos. A través de encabezados horizontales y verticales, y nodos interconectados, se logra un acceso eficiente y flexible a las calificaciones registradas. Esta elección técnica facilita operaciones como inserción, actualización y recorrido para generar visualizaciones y reportes.

b) Integración de tecnologías web: Django y Flask

La arquitectura del sistema se basa en la separación de responsabilidades mediante Django para el frontend (interfaz web y vistas) y Flask para el backend (procesamiento de XML y lógica de negocio). Esta comunicación se realiza mediante solicitudes HTTP POST con archivos XML, lo que permite modularidad y facilita el mantenimiento. Django también administra las sesiones del usuario, identificando si es un administrador, tutor o estudiante, y brindando acceso según su rol.

c) Generación y visualización de reportes académicos

Una de las funcionalidades destacadas es la generación de reportes gráficos con información relevante, como el promedio de notas por actividad o el TOP de estudiantes por actividad. Estos reportes se visualizan con herramientas como Plotly y se exportan a PDF para su análisis posterior. Además, Graphviz se utiliza para representar visualmente la estructura de la matriz de notas en formato PNG, permitiendo a los tutores inspeccionar cómo están organizados los datos internamente.

d) Carga de datos y experiencia de usuario

La plataforma permite **la carga dinámica de archivos XML que contienen horarios y notas. Este diseño simplifica la administración académica**, al permitir que los tutores suban sus datos desde un único punto, con validaciones

automáticas (por ejemplo, notas fuera del rango 0–100 son descartadas). El sistema está diseñado para mantener persistencia entre cargas, permitiendo registrar múltiples cursos y actividades sin sobrescribir la información anterior, fomentando una experiencia fluida y confiable para el usuario.

Conclusiones

El desarrollo del sistema IPC2–AcadNet ha permitido demostrar cómo el uso combinado de estructuras de datos eficientes y tecnologías web puede ofrecer soluciones funcionales, escalables y adaptadas a contextos educativos actuales. La elección de una matriz dispersa como eje central para almacenar notas académicas resalta la importancia de diseñar estructuras a medida para necesidades específicas, optimizando recursos y facilitando el análisis de datos.

La integración entre Django y Flask ha evidenciado las ventajas de una arquitectura distribuida, donde el backend puede asumir tareas complejas como la validación y el procesamiento de archivos XML, mientras el frontend se enfoca en la interacción con el usuario. El uso de herramientas como Graphviz y Plotly ha potenciado la capacidad del sistema para presentar información visualmente clara, útil para la toma de decisiones.

Como aporte principal, este proyecto ofrece una base sólida para futuras extensiones que incluyan control de versiones en las notas, analítica predictiva sobre el rendimiento estudiantil, o mecanismos de retroalimentación automatizada.

¿Es posible que una herramienta académica digital logre reducir la carga administrativa de los tutores sin sacrificar la calidad del seguimiento académico? Este tipo de preguntas abren la puerta a nuevas investigaciones y desarrollos.

Finalmente, se recomienda explorar mejoras en accesibilidad, internacionalización del sistema y

seguridad de datos como pasos siguientes en la evolución de AcadNet.

Referencias bibliográficas

Python Software Foundation. (s.f.). *The Python Tutorial*. Obtenido de <https://docs.python.org/3/>

Django Software Foundation. (s.f.). *Django Documentation*. Obtenido de <https://docs.djangoproject.com/en/stable/>

Pallets Projects. (s.f.). *Flask Documentation*. Obtenido de <https://flask.palletsprojects.com/>

Graphviz Team. (s.f.). *Graphviz - Graph Visualization Software*. Obtenido de <https://graphviz.org/documentation/>

Plotly Technologies Inc. (s.f.). *Plotly Python Graphing Library*. Obtenido de <https://plotly.com/python/>

Extensión:

Figura 1: Clases Usuario, Curso, NodoInterno, NodoCabecera

```
class Usuario:
    def __init__(self, identificador, contrasenia, nombre, tipo):
        self.id = identificador
        self.contrasenia = contrasenia
        self.nombre = nombre
        self.tipo = tipo
        self.cursos = []
        self.horarios = None
```

```
class Curso:
    def __init__(self, codigo, nombre):
        self.codigo = codigo
        self.nombre = nombre
        self.matriz_notas = None
```

```
class NodoInterno():
    def __init__(self, x, y, valor):
        self.x = x # Coordenada en x (fila)
        self.y = y # Coordenada en y (columna)
        self.valor = valor # valor del nodo interno
        # apuntadores:
        self.siguiente = None
        self.anterior = None
        self.abajo = None
        self.arriba = None
```

```
class NodoCabecera():
    def __init__(self, id):
        self.id = id
        self.siguiente = None
        self.anterior = None
        self.acceso = None
```

```
class MatrizDispersa():
    def __init__(self, capa):
        self.capa = capa
        self.filas = ListaEncabezado('fila')
        self.columnas = ListaEncabezado('columna')
```

Figura 2: Estructura de carpetas del backend utilizadas por flask

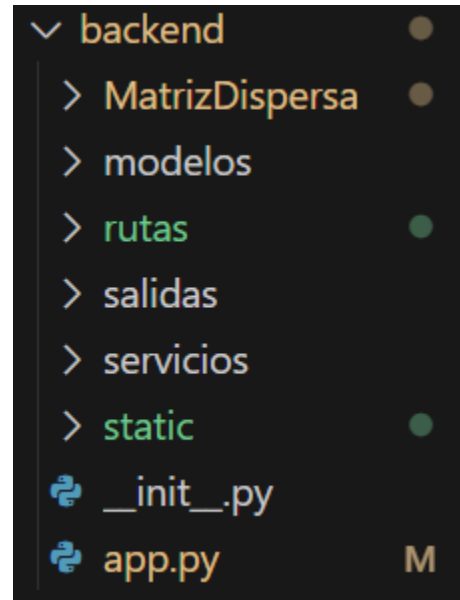


Figura 3: app.py

```
from flask import Flask
from rutas.config_rutas import config_bp
from rutas.login_rutas import login_bp
from rutas.tutor_rutas import tutor_bp
from rutas.estudiante_rutas import estudiante_bp

app = Flask(__name__)

# Registro de Blueprints
app.register_blueprint(config_bp, url_prefix='/api/config')
app.register_blueprint(login_bp, url_prefix='/api/login')
app.register_blueprint(tutor_bp, url_prefix='/api/tutor')
app.register_blueprint(estudiante_bp, url_prefix='/api/estudiante')

@app.route('/')
def home():
    return {"mensaje": "IPC2-AcadNet Backend funcionando correctamente"}

if __name__ == '__main__':
    app.run(debug=True)
```

Figura 4: Estructura de frontend utilizadas por Django

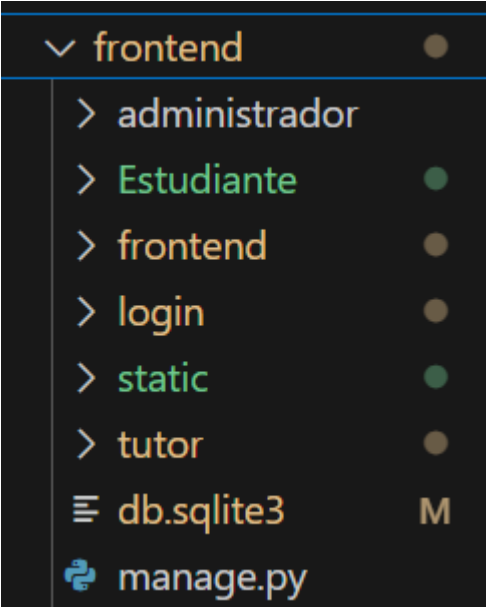


Figura 6: Diagrama de Clases del Proyecto 2

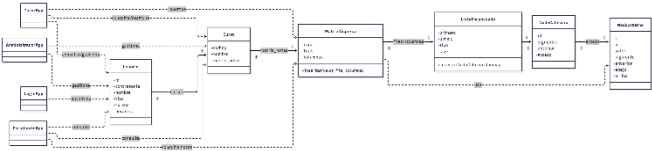


Figura 5: manage.py

```
#!/usr/bin/env python
"""Django's command-line utility for administrative tasks."""
import sys
import os
sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), '..', 'backend')))

def main():
    """Run administrative tasks."""
    os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'frontend.settings')
    try:
        from django.core.management import execute_from_command_line
    except ImportError as exc:
        raise ImportError(
            "Couldn't import Django. Are you sure it's installed and "
            "available on your PYTHONPATH environment variable? Did you "
            "forget to activate a virtual environment?"
        ) from exc
    execute_from_command_line(sys.argv)

if __name__ == '__main__':
    main()
```